

Database Design: Normalization, Indexes, Primary Keys, Foreign Keys

1. Start With a Simple Example

Suppose we have a payment system.

A naive table might look like:

```
PAYMENT
-----
payment_id
customer_id
customer_name
customer_email
payment_amount
payment_status
payment_method
```

Now imagine the same customer makes 1,000 payments.

You store:

```
customer_name
customer_email
```

1,000 times.

That's a sign that the data model may not be normalized appropriately.

A better design separates entities:

```
CUSTOMER
-----
customer_id PK
name
email

PAYMENT
-----
payment_id PK
customer_id FK
amount
status
payment_method
```

Relationship:

```
CUSTOMER
  1
  |
  | has many
  |
  N
PAYMENT
```

2. Primary Key

A **primary key (PK)** uniquely identifies each row in a table.

Example:

```
CREATE TABLE payment (  
  payment_id BIGINT PRIMARY KEY,  
  customer_id BIGINT,  
  amount DECIMAL(12,2),  
  status VARCHAR(20)  
);
```

Here:

```
payment_id
```

uniquely identifies a payment.

You cannot have:

```
payment_id = 100  
payment_id = 100
```

in the same table.

A primary key is:

- unique
 - not nullable
 - the main identifier for a row
-

3. Natural Key vs Surrogate Key

This is a useful 3-year interview topic.

Natural key

An existing business value acts as the identifier.

Example:

```
email  
PAN  
policy number
```

Surrogate key

A technical identifier is created for the row.

Example:

```
payment_id = 10001
```

Common choices:

- auto-increment integer
- UUID

In many transactional systems, surrogate keys are used because business attributes can change and are not always ideal as relational identifiers.

But a business identifier such as `transaction_id` may still need a **unique constraint** even if it isn't the primary key.

4. Primary Key vs Unique Constraint

Suppose:

```
payment_id → PK
transaction_id → UNIQUE
```

The primary key identifies the row.

The unique constraint ensures no two rows use the same transaction ID.

For a payment system, this distinction can be important:

```
payment_id
→ internal database identity

transaction_id
→ business/external transaction identifier
```

The exact design depends on the system.

5. Foreign Key

A **foreign key (FK)** establishes a relationship to another table.

Example:

```
CREATE TABLE payment (
  payment_id BIGINT PRIMARY KEY,
  customer_id BIGINT,
  amount DECIMAL(12,2),

  FOREIGN KEY (customer_id)
    REFERENCES customer(customer_id)
);
```

Here:

```
payment.customer_id
```

references:

```
customer.customer_id
```

Relationship:

```
CUSTOMER
customer_id PK
  ↑
  |
payment.customer_id FK
```

6. Why Foreign Keys?

They enforce **referential integrity**.

Suppose:

```
CUSTOMER
C101
C102
```

and you try:

```
INSERT INTO payment(customer_id)
VALUES ('C999');
```

If the foreign key is enforced and C999 doesn't exist, the database rejects the operation.

This prevents invalid relationships.

7. One-to-Many Example

One customer can have many payments:

```
Customer C101
  |
  ├── Payment P1
  ├── Payment P2
  └── Payment P3
```

Database:

```
CUSTOMER
-----
C101

PAYMENT
-----
P1 | C101
P2 | C101
P3 | C101
```

The foreign key belongs on the **many side**:

```
PAYMENT.customer_id
```

8. Normalization

Normalization is the process of structuring relational data to reduce:

- unnecessary duplication
- update anomalies
- insertion anomalies
- deletion anomalies

The commonly discussed normal forms are:

```
1NF
2NF
3NF
```

For a 3-year interview, **1NF, 2NF and 3NF** are the important ones.

9. First Normal Form — 1NF

A table should contain **atomic values** rather than repeating groups/multiple values in one column.

Bad:

customer_id	payment_methods
C101	CASH, CARD, UPI

Better:

```
CUSTOMER
C101

CUSTOMER_PAYMENT_METHOD
-----
C101 | CASH
C101 | CARD
C101 | UPI
```

The idea is:

One field should represent one value rather than a collection of values.

10. Second Normal Form — 2NF

2NF applies primarily when you have a **composite primary key**.

Requirements:

1. Be in 1NF.
2. Every non-key attribute must depend on the **whole primary key**, not just part of it.

Example:

```
PAYMENT_ITEM
-----
payment_id
product_id
product_name
quantity
```

Suppose the primary key is:

```
(payment_id, product_id)
```

quantity depends on both:

```
payment_id + product_id
```

But:

```
product_name
```

depends only on:

```
product_id
```

That's a partial dependency.

So split:

```
PRODUCT
-----
product_id
product_name

PAYMENT_ITEM
-----
payment_id
product_id
quantity
```

11. Third Normal Form — 3NF

3NF removes **transitive dependencies**.

Suppose:

```
EMPLOYEE
-----
employee_id
department_id
department_name
```

Here:

```
employee_id
  ↓
department_id
```

```
↓  
department_name
```

`department_name` isn't directly dependent on the employee; it depends on `department_id`.

Split:

```
EMPLOYEE  
-----  
employee_id  
department_id  
  
DEPARTMENT  
-----  
department_id  
department_name
```

Now the dependency is cleaner.

12. Why Normalize?

Imagine:

```
PAYMENT  
-----  
payment_id | customer_id | customer_name | email
```

Customer email changes.

Without normalization:

```
1000 rows  
↓  
must update many rows
```

You can accidentally end up with:

```
customer_name = Rahul  
email = old@email.com
```

in some rows and:

```
customer_name = Rahul  
email = new@email.com
```

in others.

Normalization reduces this problem.

13. Database Anomalies

Interviewers often ask about these.

Update anomaly

Same data exists in many places and must be updated everywhere.

Insert anomaly

You can't insert one piece of information without another unrelated piece.

Delete anomaly

Deleting one record accidentally removes information you still need.

Normalization helps prevent these anomalies.

14. Over-Normalization

Don't assume:

"More normalization is always better."

Too much normalization can lead to:

```
Table A
  ↓
Table B
  ↓
Table C
  ↓
Table D
  ↓
Table E
```

Now simple queries require many joins.

Potential consequences:

- complex queries
- more joins
- harder reporting
- potentially higher query latency

This is why production database design sometimes uses **controlled denormalization**.

15. Denormalization

Denormalization intentionally duplicates some data for performance or query simplicity.

Example:

Instead of joining:

```
ORDER
+
CUSTOMER
```

you might store:

```
ORDER
-----
order_id
customer_id
customer_name
```

even though `customer_name` already exists in CUSTOMER.

Why?

Maybe the application has extremely high read volume and wants a simpler/faster read path.

But now you have a consistency problem:

```
Customer name changes
      ↓
Order copies may need updating
```

So denormalization is a trade-off.

16. Indexes

An **index** is a data structure that helps the database locate rows efficiently without scanning the entire table.

Suppose:

```
PAYMENT
10 million rows
```

Query:

```
SELECT *
FROM payment
WHERE transaction_id = 'TX123';
```

Without a suitable index, the database may need to inspect many rows.

With:

```
CREATE INDEX idx_payment_transaction_id
ON payment(transaction_id);
```

the database can potentially find matching rows much more efficiently.

17. Why Indexes Improve Read Performance

Think of a book.

Without index:

```
Search word
→ scan every page
```

With index:

```
Search word  
→ index  
→ page
```

Database indexes provide a similar concept, though the actual structure and access path depend on the database.

18. Indexes Are Not Free

This is **very important**.

Indexes improve some reads but cost:

- storage
- insert/update/delete work
- maintenance
- memory/cache capacity

Suppose:

```
PAYMENT  
10 million rows  
  
Indexes:  
transaction_id  
customer_id  
status  
amount  
created_at  
...
```

Now every insert may require updating several indexes.

So:

Don't index every column.

19. Which Columns Should Be Indexed?

Good candidates often include columns used frequently for:

- filtering
- joining
- sorting
- uniqueness

Examples:

```
WHERE transaction_id = ?
```

```
WHERE customer_id = ?
```

```
JOIN payment p  
ON p.customer_id = c.customer_id
```

But index choice depends on query patterns, cardinality/selectivity, data distribution and the database optimizer.

20. Index Selectivity

Suppose:

```
status
```

contains only:

```
SUCCESS  
FAILED  
PENDING
```

If 90% of rows are:

```
SUCCESS
```

an index only on `status` may not be very selective.

Compare:

```
transaction_id
```

where nearly every value is unique.

That tends to be a much more selective index candidate.

However, whether the database actually uses an index depends on the query and execution plan.

21. Composite Index

Suppose queries frequently look like:

```
SELECT *  
FROM payment  
WHERE customer_id = ?  
AND status = ?;
```

A composite index could be:

```
CREATE INDEX idx_payment_customer_status  
ON payment(customer_id, status);
```

This is an index on multiple columns.

22. Column Order Matters

For:

```
(customer_id, status)
```

the order matters because the database organizes the index according to that key ordering.

A query using:

```
WHERE customer_id = ?
```

can generally benefit.

But a query using only:

```
WHERE status = ?
```

may not benefit from that composite index in the same way.

This is related to the **leftmost-prefix principle** for common B-tree composite indexes.

Don't simply memorize:

"Composite index works for both columns."

The column order and query shape matter.

23. Example

Suppose:

```
CREATE INDEX idx_payment_customer_status  
ON payment(customer_id, status);
```

Likely useful:

```
WHERE customer_id = 'C101'
```

and:

```
WHERE customer_id = 'C101'  
AND status = 'SUCCESS'
```

Potentially less useful:

```
WHERE status = 'SUCCESS'
```

because the first indexed column isn't being constrained.

Exact optimizer behavior can vary by database.

24. Primary Key and Index

Most relational databases automatically create or maintain a unique index-like access structure for a primary key.

So:

```
PRIMARY KEY(payment_id)
```

normally makes lookup by `payment_id` efficient.

However, don't assume every database implements primary-key indexing identically; the logical guarantee is uniqueness/non-null, while the physical implementation is database-specific.

25. Foreign Key and Index

A foreign key does **not universally mean an index is automatically created on the child column**.

For example:

```
payment.customer_id
```

may benefit from an index if you frequently do:

```
WHERE customer_id = ?
```

or:

```
JOIN customer ...
```

So the common practice is to evaluate indexing of foreign-key columns based on workload and database behavior.

26. Index and UPDATE/DELETE

Indexes are not just for SELECT.

Suppose:

```
UPDATE payment
SET status = 'SUCCESS'
WHERE transaction_id = 'TX123';
```

An index on `transaction_id` can help locate the row.

But if you're updating an indexed column:

```
UPDATE payment
SET transaction_id = ...
```

the index itself must also be maintained.

That's another reason indexes have a write cost.

27. Clustered vs Non-Clustered Index

This depends heavily on the database, but conceptually:

Clustered

The table's data is organized around the index key.

Non-clustered

The index is a separate structure that points toward the underlying rows/data.

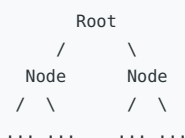
In some databases such as SQL Server, clustered indexing is a prominent explicit concept. In PostgreSQL, the physical/index behavior is different, so don't present "clustered index" as a universal relational-database rule.

For a general 3-year Java interview, understand the concept but prioritize **B-tree indexes, composite indexes and execution plans**.

28. B-Tree Index

Most traditional relational indexes use variants of **B-tree/B+tree structures**.

Conceptually:



This allows efficient searching without scanning every row.

Typical lookup complexity is often described as approximately:

```
O(log n)
```

for a B-tree-style lookup, though actual performance depends on cache, selectivity, query plan and other factors.

29. Hash Index

Some databases also support hash-based indexes.

Conceptually:

```
key
↓
hash
↓
bucket
↓
row
```

They can be useful for equality lookups.

But don't generalize that hash indexes are always faster. Range queries such as:

```
WHERE amount > 1000
```

benefit from ordered structures such as B-tree much more naturally.

30. How Does the Database Decide Whether to Use an Index?

The database **query optimizer** evaluates possible execution plans.

It may choose:

```
Index Scan
```

instead of:

```
Full Table Scan
```

but it isn't required to use an index simply because one exists.

For example, if the query matches a huge percentage of the table, scanning the table may be cheaper.

This is why:

"I created an index, therefore the query is fast" is not a sufficient explanation.

31. EXPLAIN

When a query is slow, use the database's execution-plan tools.

Examples:

```
EXPLAIN
SELECT *
FROM payment
WHERE transaction_id = 'TX123';
```

Depending on the database, you may have:

This helps investigate:

- scan type
- index usage
- estimated rows
- actual rows
- join strategy
- cost
- execution time

This is an important next-level SQL skill.

32. Primary Key + Foreign Key + Index Together

Consider:

```
CUSTOMER
-----
customer_id PK
name
email

PAYMENT
-----
payment_id PK
customer_id FK
amount
status
```

Indexes could be:

```
CUSTOMER
  PK(customer_id)

PAYMENT
  PK(payment_id)
  INDEX(customer_id)
```

Then:

```
SELECT p.payment_id, p.amount
FROM payment p
WHERE p.customer_id = 'C101';
```

can benefit from the `customer_id` index.

33. A Realistic Payment Schema

You can mentally model something like:

```
CUSTOMER
```

```

-----
customer_id PK
name
email

PAYMENT
-----
payment_id PK
transaction_id UNIQUE
customer_id FK
amount
status
vendor
created_at

PAYMENT_ATTEMPT
-----
attempt_id PK
payment_id FK
vendor
status
response_code
created_at

```

Relationship:

```

Customer
|
| 1:N
↓
Payment
|
| 1:N
↓
PaymentAttempt

```

This gives you a concrete mental model for the interview without assuming it's your actual BPS schema.

34. How an Interviewer Might Challenge You

Question:

"Why is `transaction_id` indexed?"

Weak answer:

"Because indexes improve performance."

Better:

"The application frequently retrieves transactions using `transaction_id`, so an index avoids unnecessary scanning and makes that lookup more efficient. Since transaction IDs are expected to be unique, a unique constraint/index may also enforce the business rule."

Question:

"Why not index status?"

Good answer:

"Status has relatively low cardinality, so an index on status alone may have limited benefit depending on the data distribution and query workload. I'd check the execution plan and consider whether a composite index matching the actual query pattern would be more useful."

That is a much stronger 3-year answer.

35. Normalization + Indexing Are Different

Don't mix them.

Normalization

How should data be structured?

Indexing

How should data be accessed efficiently?

For example:

```
Normalization
→ split CUSTOMER and PAYMENT

Indexing
→ index PAYMENT.transaction_id
```

36. Interview Questions

Primary/Foreign Keys

1. What is a primary key?
2. What is a foreign key?
3. Primary key vs unique key?
4. Can a primary key be NULL?
5. Can a foreign key be NULL?
6. Why use foreign keys?
7. What is referential integrity?
8. One-to-one, one-to-many, many-to-many?

9. Where should the foreign key go in a one-to-many relationship?
10. What is a composite primary key?

Normalization

11. What is normalization?
12. Explain 1NF.
13. Explain 2NF.
14. Explain 3NF.
15. What is partial dependency?
16. What is transitive dependency?
17. What are update/insert/delete anomalies?
18. What is denormalization?
19. When would you intentionally denormalize?

Indexes

20. What is an index?
21. Why does an index improve query performance?
22. What are the disadvantages of indexes?
23. What columns should be indexed?
24. What is a composite index?
25. Why does column order matter?
26. What is index selectivity?
27. B-tree vs hash index?
28. What is a covering index?
29. Can too many indexes hurt performance?
30. Why isn't an index always used?
31. How do you verify whether an index is being used?
32. What is **EXPLAIN** ?

37. High-Probability 3-Year Questions

These are the ones I'd practice until you can answer naturally:

Q: Why not create an index on every column?

"Indexes consume storage and add overhead to inserts, updates and deletes because the index structures must also be maintained. They should be created based on actual query patterns and verified using execution plans."

Q: Composite index **(customer_id, status)** vs separate indexes?

"It depends on the workload. A composite index can efficiently support queries using the leading column and combinations matching the index order, while separate indexes may offer different flexibility. I'd decide based on the actual query patterns, cardinality and execution plans."

Q: Why normalize a database?

"To reduce unnecessary duplication and prevent update, insert and delete anomalies while maintaining clear data ownership and relationships."

Q: Why denormalize?

"To optimize specific read-heavy workloads or simplify expensive query paths when the performance benefit justifies the additional consistency and storage complexity."

38. What You Should Inspect in Your BPS Codebase

Now relate this to your actual system yourself:

1. What are the main BPS tables?
 2. What is the primary key of each?
 3. Which columns are business identifiers?
 4. Which columns have unique constraints?
 5. What foreign-key relationships exist?
 6. Which tables represent one-to-many relationships?
 7. Are there composite keys?
 8. Which columns are indexed?
 9. Why is each index present?
 10. Which queries are executed frequently?
 11. Does a query use a composite index?
 12. Can you run `EXPLAIN` on one important BPS query?
 13. Are there cases where normalization created many joins?
 14. Are any fields deliberately denormalized?
 15. How would you prove that an index actually improved performance?
-

Interview-ready summary

"Primary keys uniquely identify rows, while foreign keys establish relationships and enforce referential integrity between tables. Normalization structures relational data to reduce duplication and anomalies; for a typical backend interview, I would know 1NF, 2NF and 3NF and understand when controlled denormalization may be justified. Indexes are data structures that allow the database to locate rows more efficiently for suitable queries, but they consume storage and add write-maintenance cost. Indexes should therefore be designed around actual access patterns. Composite indexes are particularly important because column order affects which queries can benefit. Finally, when performance matters, I would use an execution plan such as `EXPLAIN` / `EXPLAIN ANALYZE` rather than assuming an index is being used."

The mental model

Primary Key

→ Who is this row?

Foreign Key

→ How is this row related to another table?

Normalization

→ How should the data be structured?

Index

→ How can the database find the data efficiently?

For your **3-year backend preparation**, the next database topic should be **Transactions + ACID + isolation levels + optimistic/pessimistic locking**, because these naturally build on primary keys, foreign keys and indexing and are especially important for transactional/payment systems.